



Fundamentals of Accelerated Data Science

NVIDIA Deep Learning Institute

What This Course Will Cover

An introduction to data science with a focus on speed and efficiency

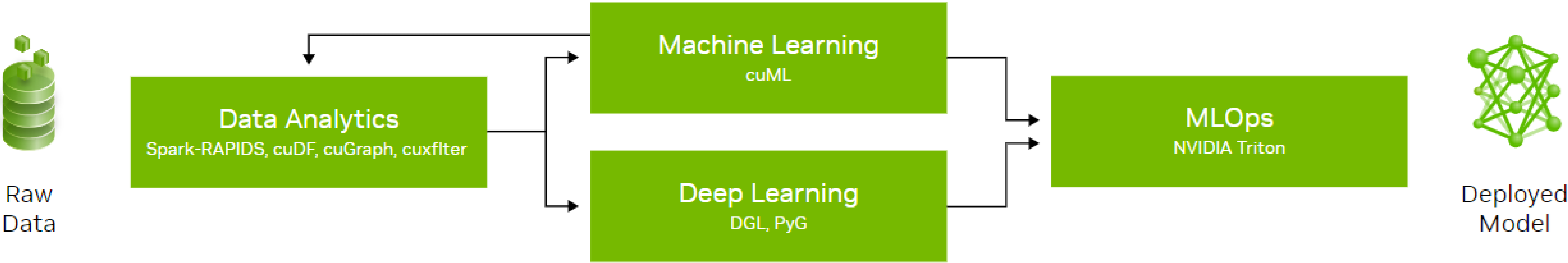
Learning objectives:

- Overview of data science
- Demonstrations of accelerated data science pipelines
- How acceleration is achieved and its implications

This workshop will not cover:

- Neural networks (accelerated by cuDNN)
- Statistical analysis

	Course Outline
Task 1	Data science overview Data manipulation
Task 2	Machine learning
Task 3	Graph analytics
Coding Assessment	Biodefense





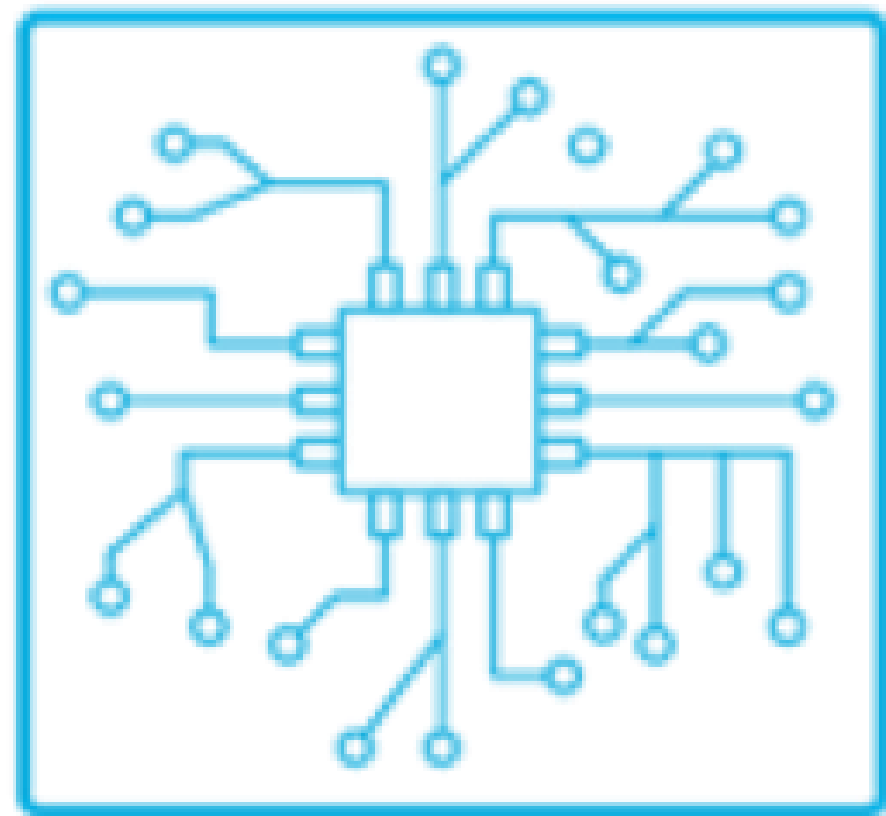
Agenda

- Machine Learning Fundamentals
- End-To-End Machine Learning
- Accelerating Machine Learning
- Hands-On Lab

Machine Learning

Data, algorithm, and compute

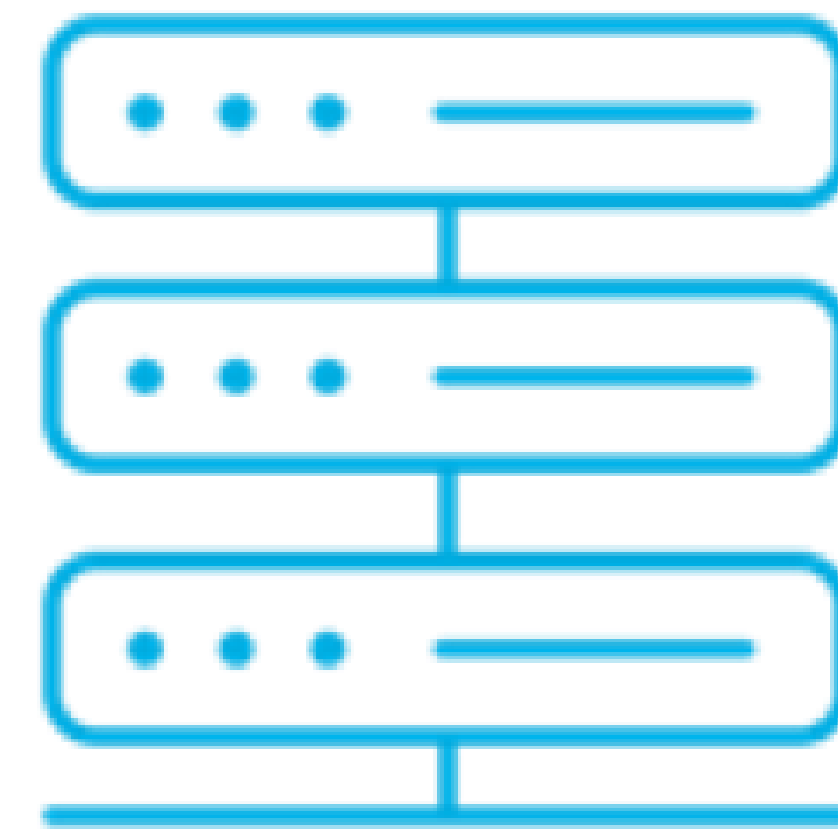
- Machine learning primarily focuses on building algorithms that enable computers to learn from data and make predictions. In contrast, data science has a broader focus that encompasses various techniques for extracting insights and meaning from data, including statistical analysis and data visualization.
- Pillars of machine learning:



Computing Power



Algorithm Power

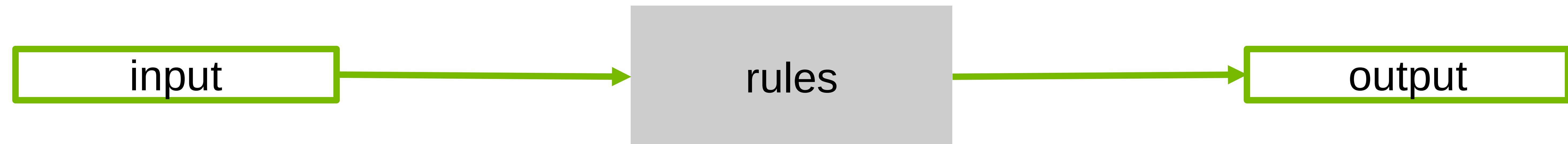


Data Availability

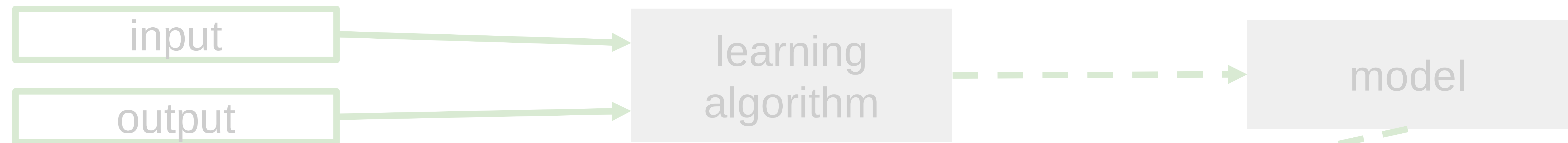
Traditional Programming: Rules-Based

Explicitly define rules and logic to follow

Rules-based



Training



Inference



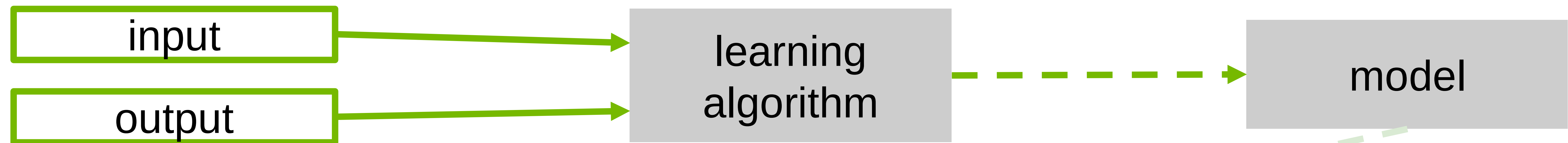
Model Training

Models learn patterns from data to make predictions or decisions without explicit programming

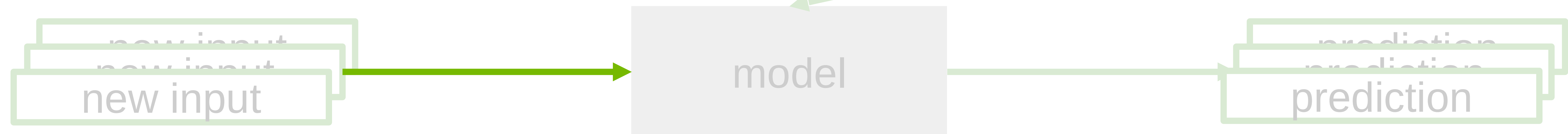
Rules-based



Training



Inference



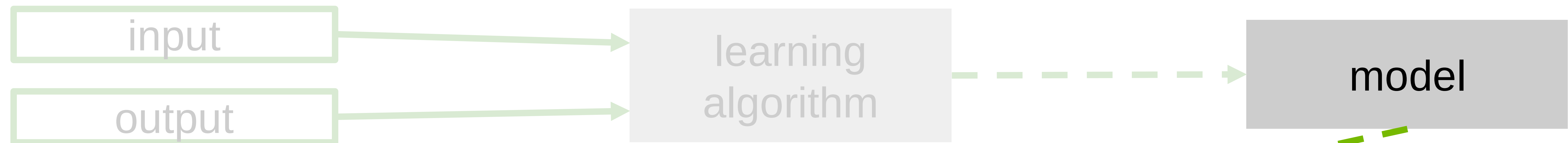
Inference

Process incoming data and producing results based on patterns learned during training

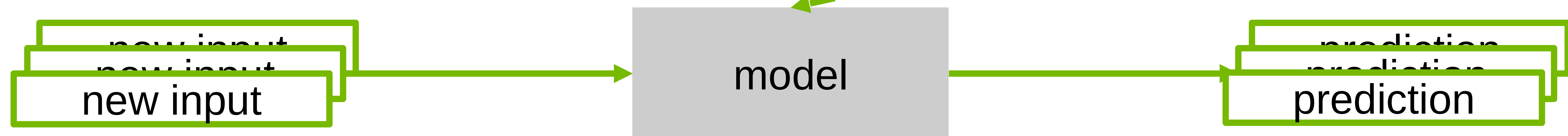
Rules-based



Training



Inference



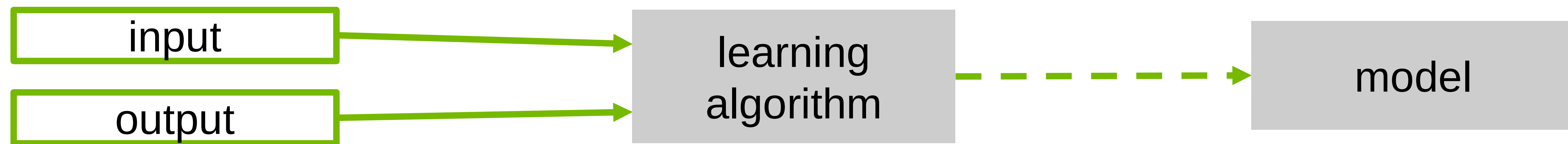
Supervised vs. Unsupervised

Trained on labeled data to make predictions or classifications on new, unseen data

Rules-based



Training



Supervised:

- **Regression:** predicting a continuous value
- **Classification:** predicting a discrete label

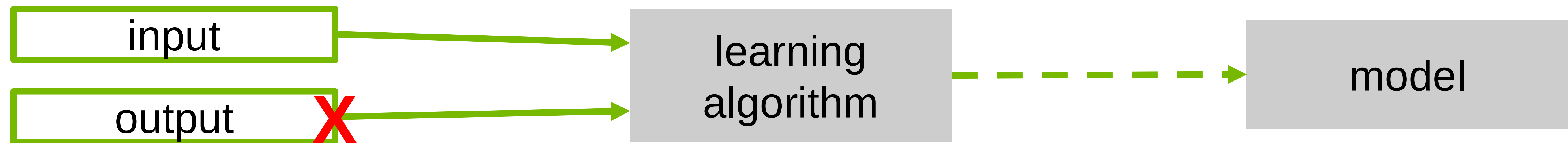
Supervised vs. Unsupervised

Identifying unknown patterns and structures in data without predefined categories

Rules-based



Training



Supervised:

- **Regression:** predicting a continuous value
- **Classification:** predicting a discrete label

Unsupervised:

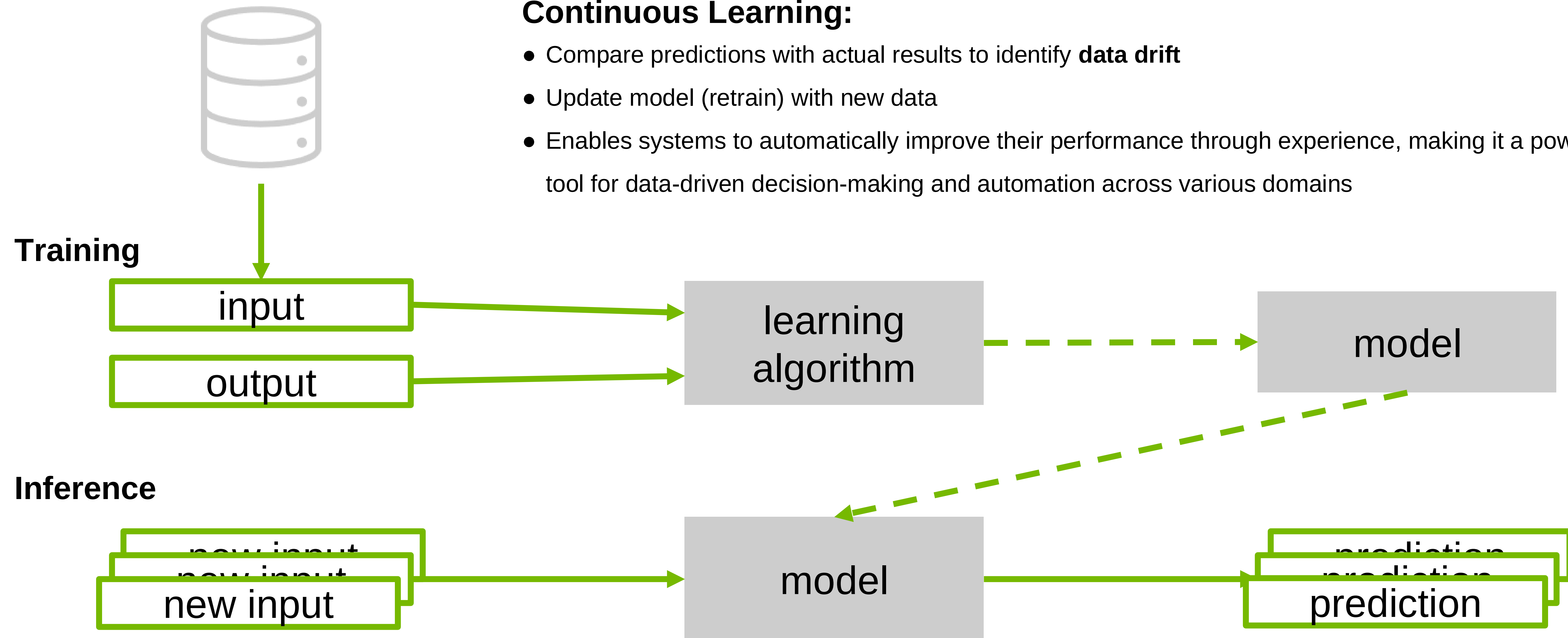
- **Clustering:** grouping data points together
- **Dimensionality reduction** while retaining information

Lifelong Learning

Allows models to update and improve their performance as new data becomes available

Continuous Learning:

- Compare predictions with actual results to identify **data drift**
- Update model (retrain) with new data
- Enables systems to automatically improve their performance through experience, making it a powerful tool for data-driven decision-making and automation across various domains



Common Use Cases

	Supervised	Unsupervised
Online Services	• Spam / Not Spam • Predict Probability Customer Will Click on Ad	• Group Articles into Categories • Similar Search Results
Financial Services	• Credit Card Fraud	• Anomaly Detection
Telecom	• Predict Customer Churn	
Health Care	• Probability of Readmission • Predict Days of Hospital Stay	• Patient Similarity
Real Estate	• Predict House Price	
Retail	• Prejudice Price • Forecast Sales • Sentiment Analysis	• Similar Customers • Product Similarity • Customer Group • Products Which Are Purchased Together

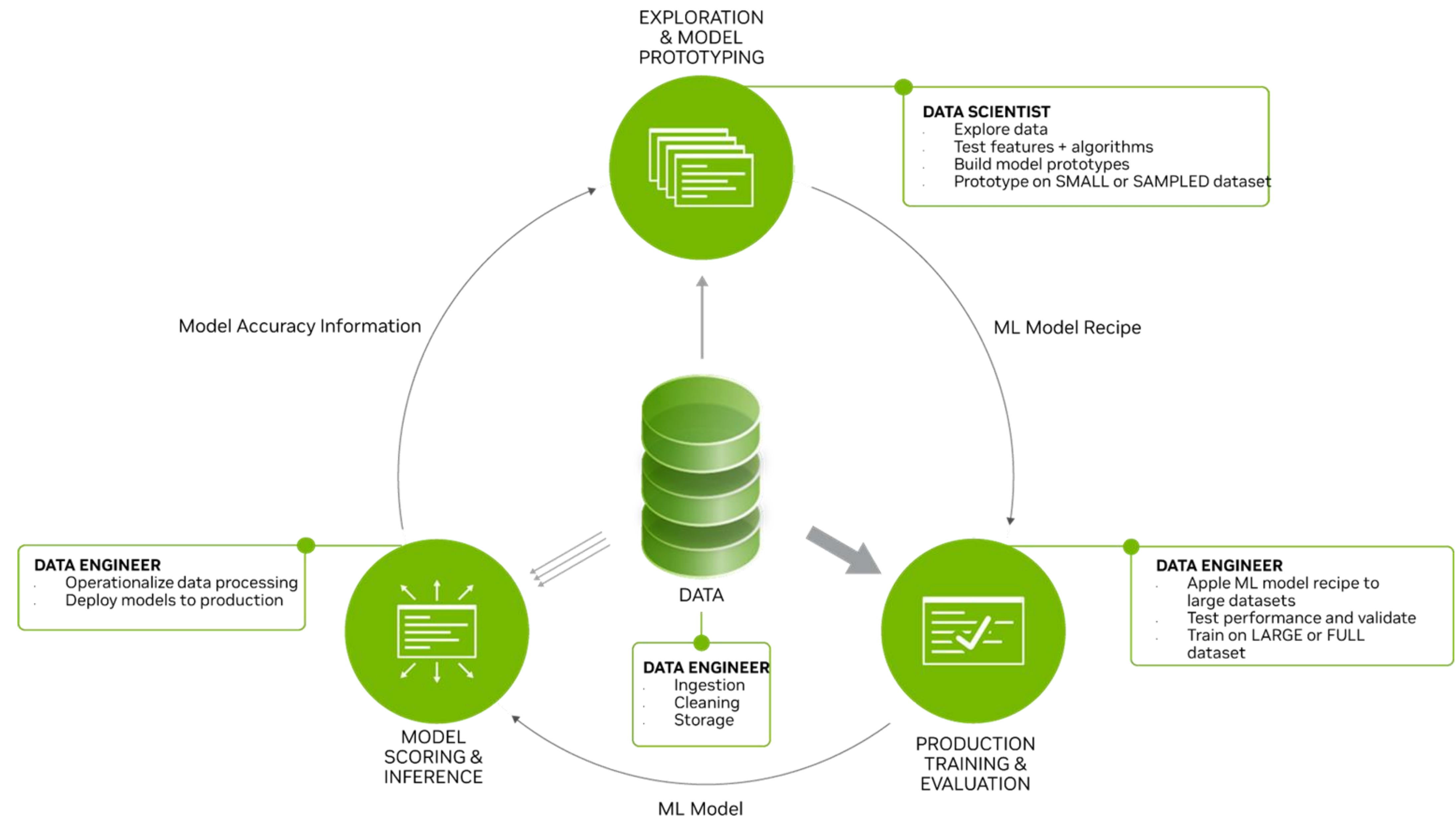
Benefits:

- **Adaptability:** models can adapt to new data, new tasks, and improve over time with retraining
- **Problem solving:** excels at complex problems where patterns are not easily discernible

Machine Learning Lifecycle

Model development-> evaluation -> deployment

- Machine learning is not a linear process
- Benefits of accelerated data science workflows:
 - Higher productivity
 - More explorations and iterations can improve model accuracy and optimize inference performance
 - Lower time to insight and total cost of ownership



Model Evaluation

Assessing the quality and performance of a trained machine learning model

Accuracy

- Assessing the performance and quality of a trained machine learning model using different metrics and techniques
- To understand how well the model generalizes to unseen data and makes accurate predictions:
 - Underfit
 - Overfit
 - Data leakage
- Employing appropriate metrics based on the type of problem:
 - Classification: **accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix**
 - Regression: **R-squared, MSE, RMSE, MA**
- Using techniques likes:
 - Cross-validation
 - Holdout validation
 - Bootstrapping

Model Evaluation

Assessing the quality and performance of a trained machine learning model

- **Latency:** the time it takes for a model to generate predictions or outputs when given input data
- **Throughput:** the amount of outputs it can perform in a given time, which is important for applications handling large volumes of data
- Ensuring that optimizations for speed don't significantly degrade the model's accuracy or performance on its intended task
- Evaluating how well the inference performance scales with increasing data volumes or concurrent requests

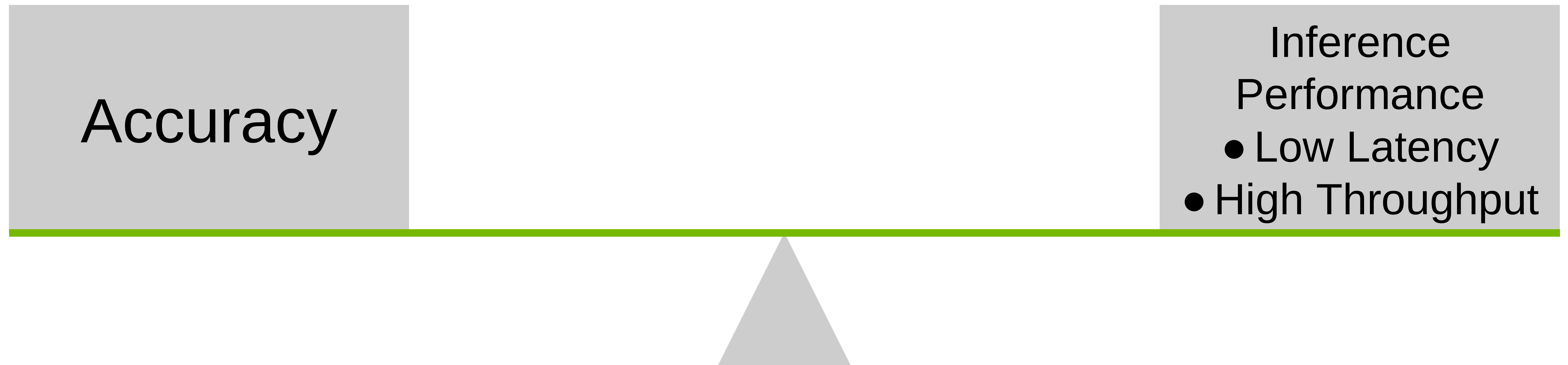
**Inference
Performance**

- Low Latency
- High Throughput

Model Evaluation

Finding the balance between speed and accuracy

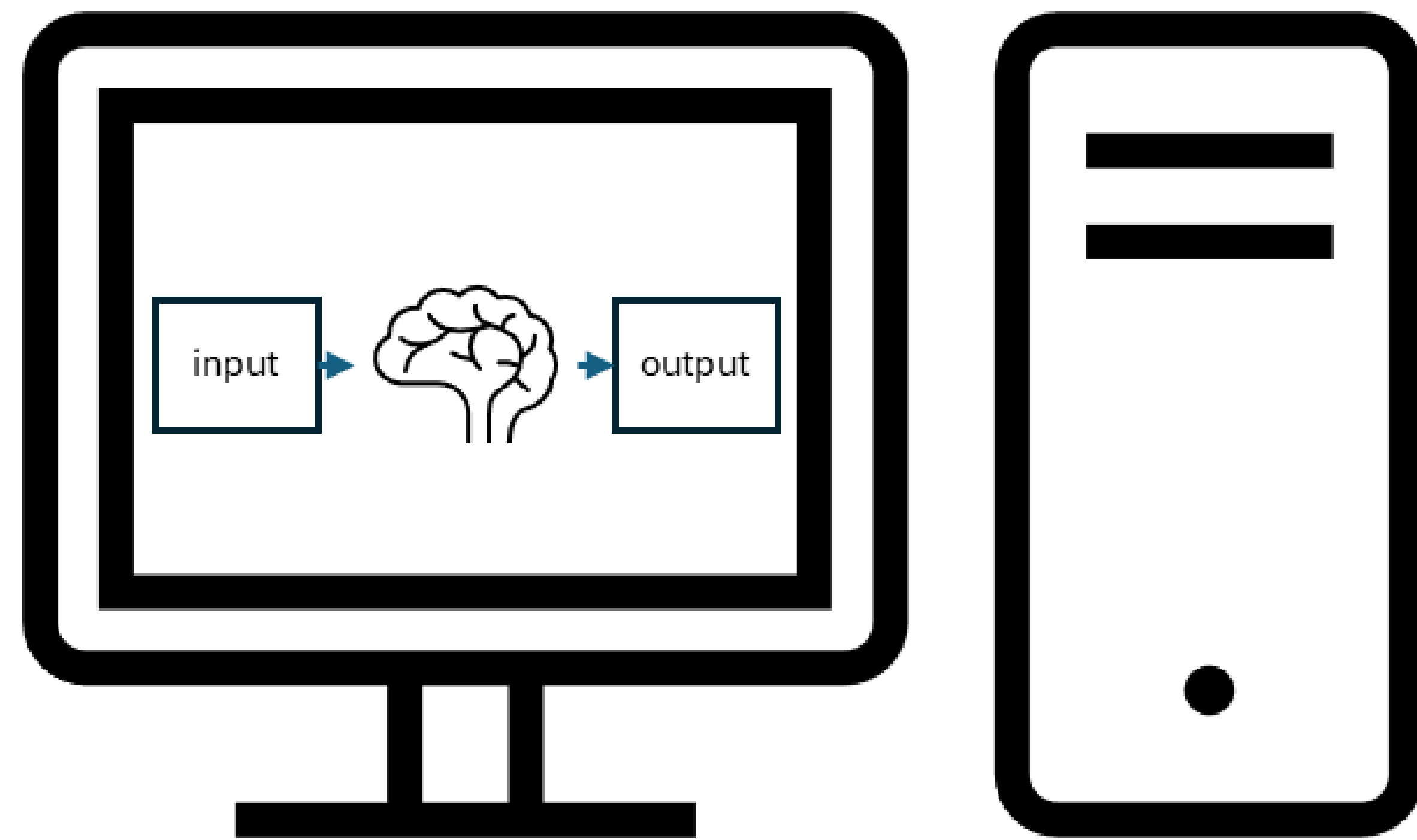
Larger and More Complex Models = Higher Accuracy = Lower Inference Performance



Model Deployment

On-device vs. server-based

On-Device Inference



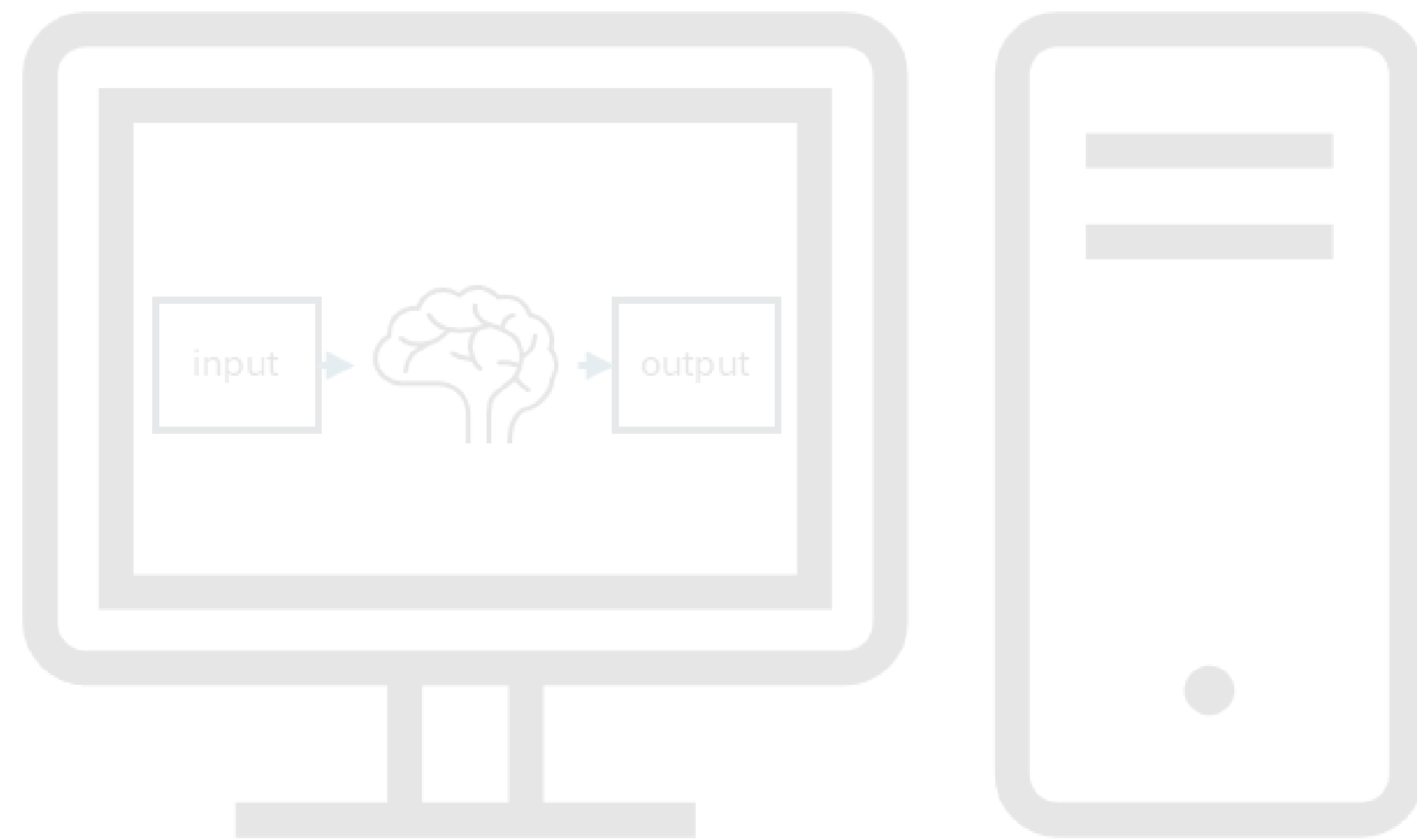
Server-Based Inference



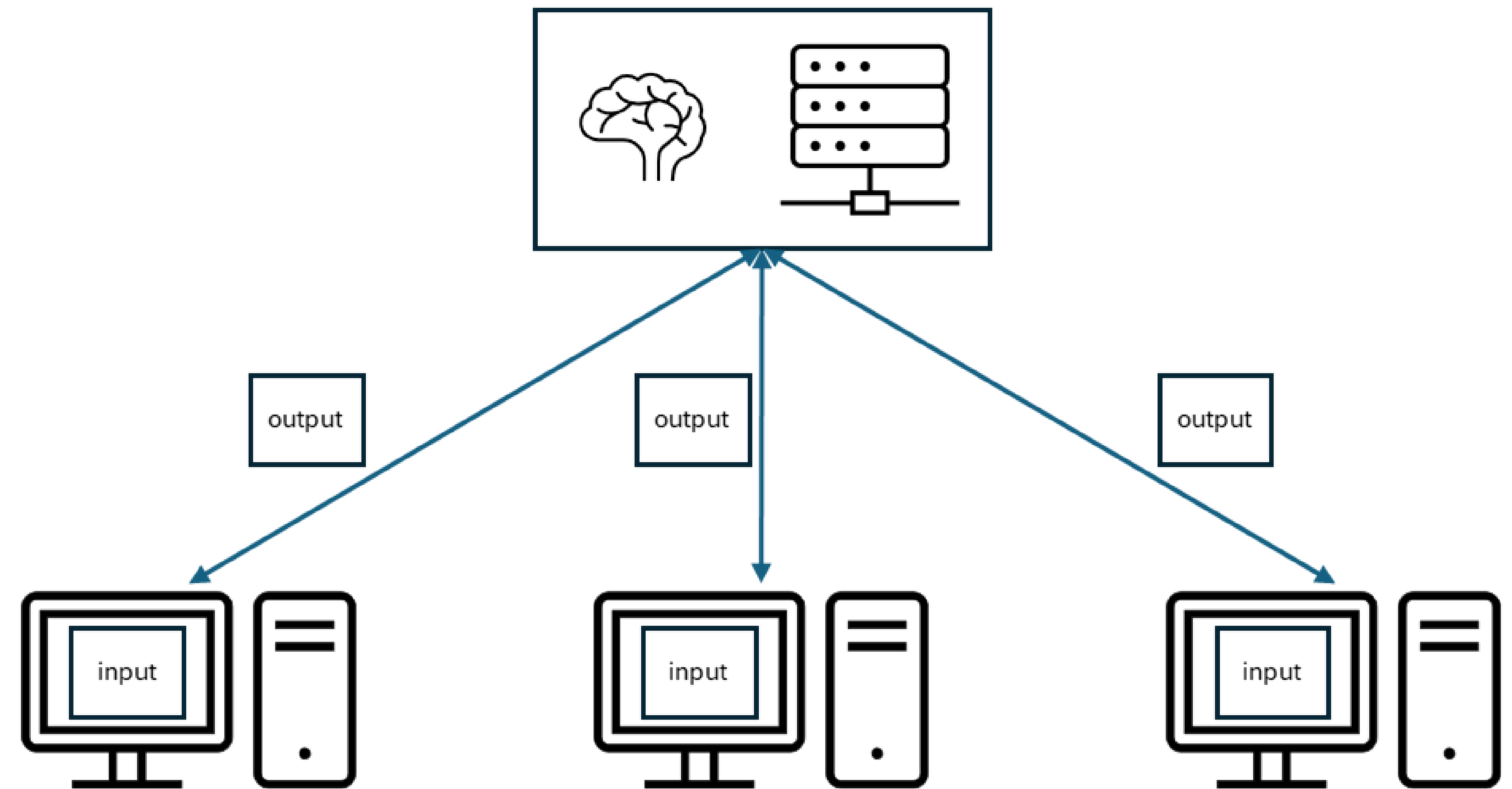
Model Deployment

On-device vs. server-based

On-Device Inference



Server-Based Inference



Triton Inference Server

Open-source inference serving software that is designed for deploying AI models at scale

Key Features:

- Multi-framework support (TensorFlow, PyTorch, ONNX, etc.)
- Concurrent model execution
- Dynamic batching for improved performance
- Model versioning and ensembling

Benefits for Data Scientists:

- Simplifies model deployment in production environments
- Enables efficient scaling of machine learning models
- Provides a consistent interface across different ML frameworks

Triton Inference Server - Performance Analyzer

Command-line tool for benchmarking and analyzing Triton server performance

Key Capabilities

- Measures throughput, latency, and other performance metrics
- Supports various input data types and shapes
- Allows customization of test parameters (batch size, concurrency, etc.)

Benefits:

- Optimizes model serving configurations
- Identifies performance bottlenecks
- Enables data-driven decisions for deployment strategies

```
$ perf_analyzer -m inception_graphdef --concurrency-range 1:4
```

```
*** Measurement Settings ***
```

```
Batch size: 1
```

```
Measurement window: 5000 msec
```

```
Latency limit: 0 msec
```

```
Concurrency limit: 4 concurrent requests
```

```
Using synchronous calls for inference
```

```
Stabilizing using average latency
```

```
Request concurrency: 1
```

```
Client:
```

```
Request count: 339
```

```
Throughput: 67.8 infer/sec
```

```
Avg latency: 14710 usec (standard deviation 2539 usec)
```

```
p50 latency: 13665 usec
```

```
...
```

```
Request concurrency: 4
```

```
Client:
```

```
Request count: 415
```

```
Throughput: 83 infer/sec
```

```
Avg latency: 48064 usec (standard deviation 6412 usec)
```

```
p50 latency: 47975 usec
```

```
...
```

```
Inferences/Second vs. Client Average Batch Latency
```

```
Concurrency: 1, throughput: 67.8 infer/sec, latency 14710 usec
```

```
Concurrency: 2, throughput: 89.8 infer/sec, latency 22280 usec
```

```
Concurrency: 3, throughput: 80.4 infer/sec, latency 37283 usec
```

```
Concurrency: 4, throughput: 83 infer/sec, latency 48064 usec
```


Triton Inference Server - Model Analyzer

Command-line tool for optimizing model configurations on Triton

Features:

- Automatically finds optimal configurations for models
- Supports single, multiple, ensemble, and BLS models
- Measures performance metrics like throughput, latency, and GPU utilization
- Generates detailed reports and visualizations

Search Modes:

- Quick Search: Uses heuristic algorithm for faster results
- Automatic Brute Search: Exhaustive search of configuration parameters
- Manual Brute Search: Allows custom parameter sweeps

Benefits:

- Improves model serving performance
- Saves development time
- Maximizes hardware utilization
- Helps meet application constraints (latency, throughput, memory)

Example of Model Analyzer Report

Online Result Summary

Model: resnet50_libtorch

GPU(s): 1 x NVIDIA TITAN RTX

Total Available GPU Memory: 23.7 GB

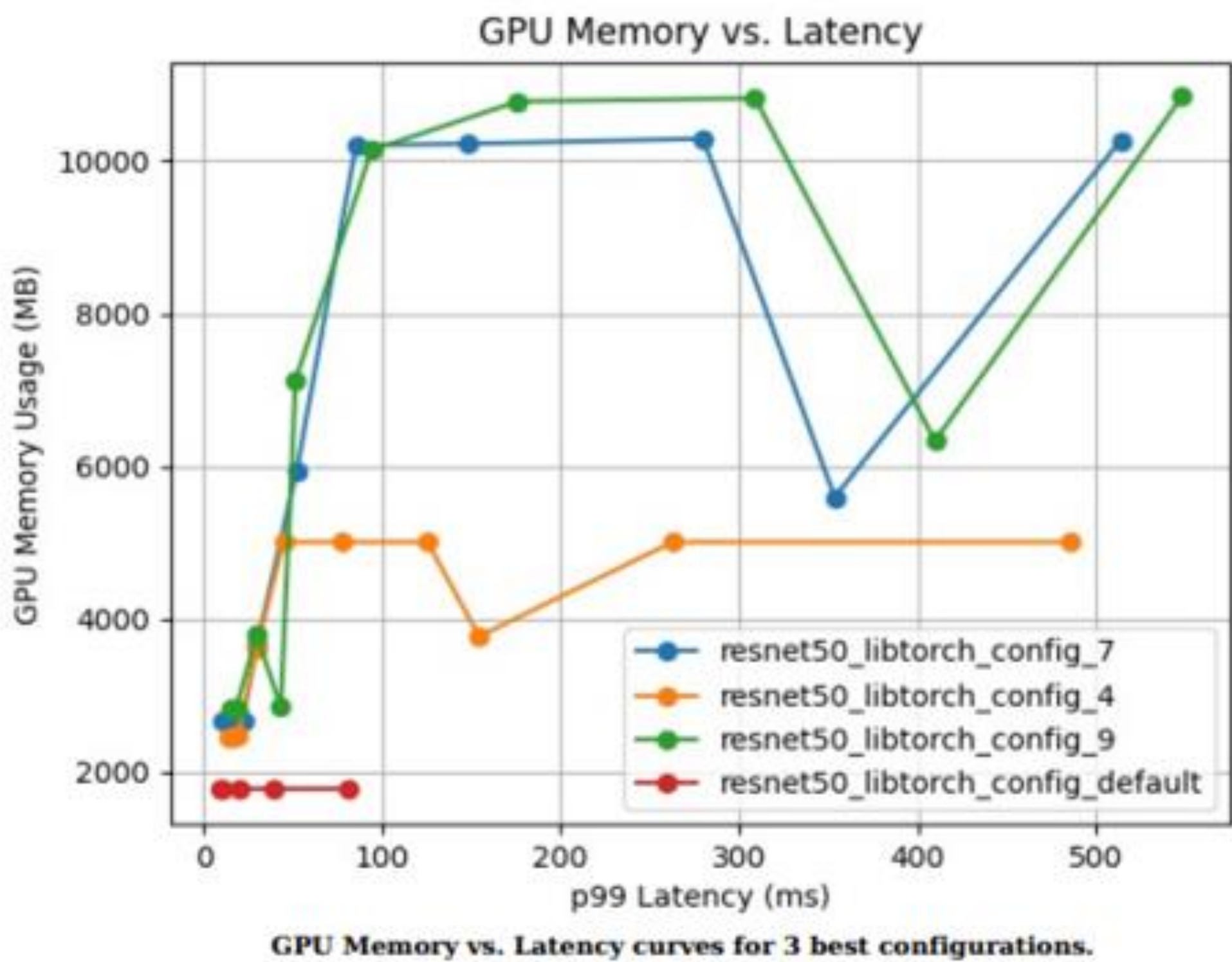
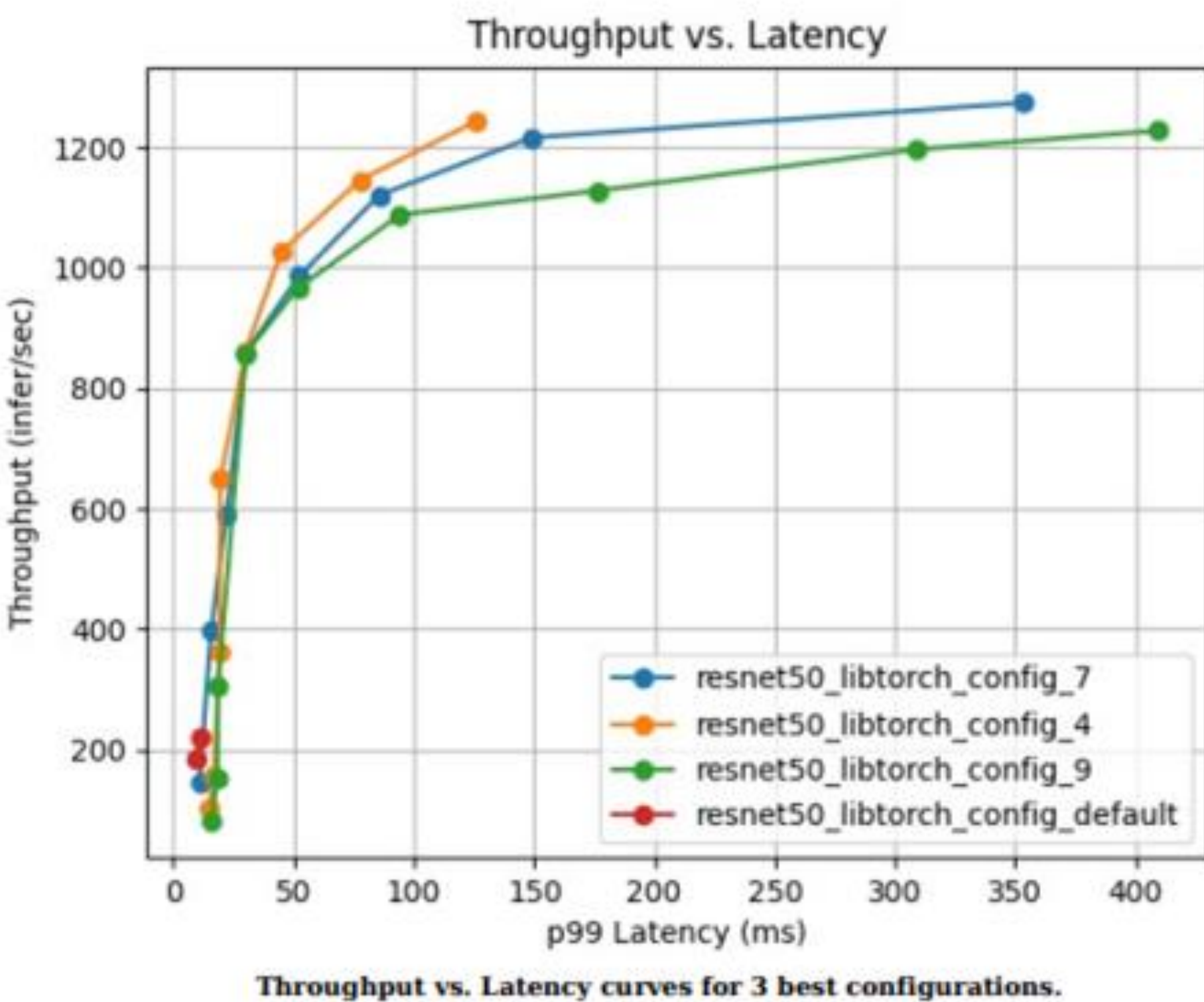
Constraint targets: None

In 46 measurements across 12 configurations, **resnet50_libtorch_config_7** provides the best throughput: **1273 infer/sec**.

This is a **473% gain** over the default configuration (222 infer/sec), under the given constraints on GPU(s) 1 x NVIDIA TITAN RTX.

- **resnet50_libtorch_config_7**: 6 GPU instances with a max batch size of 32 on platform pytorch_libtorch

Curves corresponding to the 3 best model configuration(s) out of a total of 12 are shown in the plots.



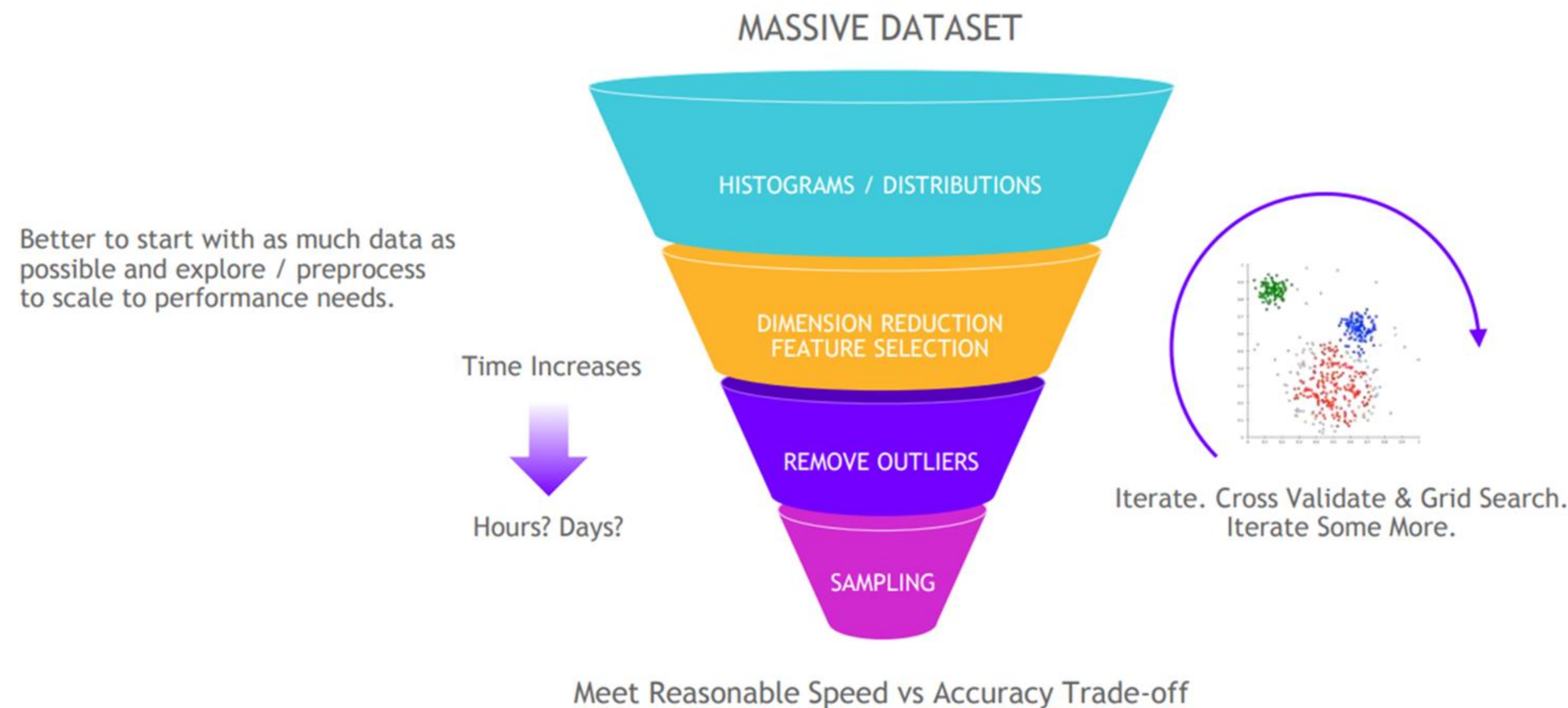
The following table summarizes each configuration at the measurement that optimizes the desired metrics under the given constraints.

Model Config Name	Max Batch Size	Dynamic Batching	Total Instance Count	p99 Latency (ms)	Throughput (infer/sec)	Max GPU Memory Usage (MB)	Average GPU Utilization (%)
resnet50_libtorch_config_7	32	Enabled	6:GPU	353.539	1273.38	5598	94.3
resnet50_libtorch_config_4	16	Enabled	5:GPU	125.635	1242.25	5011	90.2
resnet50_libtorch_config_9	32	Enabled	7:GPU	409.616	1227.02	6334	95.3
resnet50_libtorch_config_default	128	Disabled	1:GPU	10.938	221.846	1773	41.0

Challenges in Machine Learning

More data requires more compute

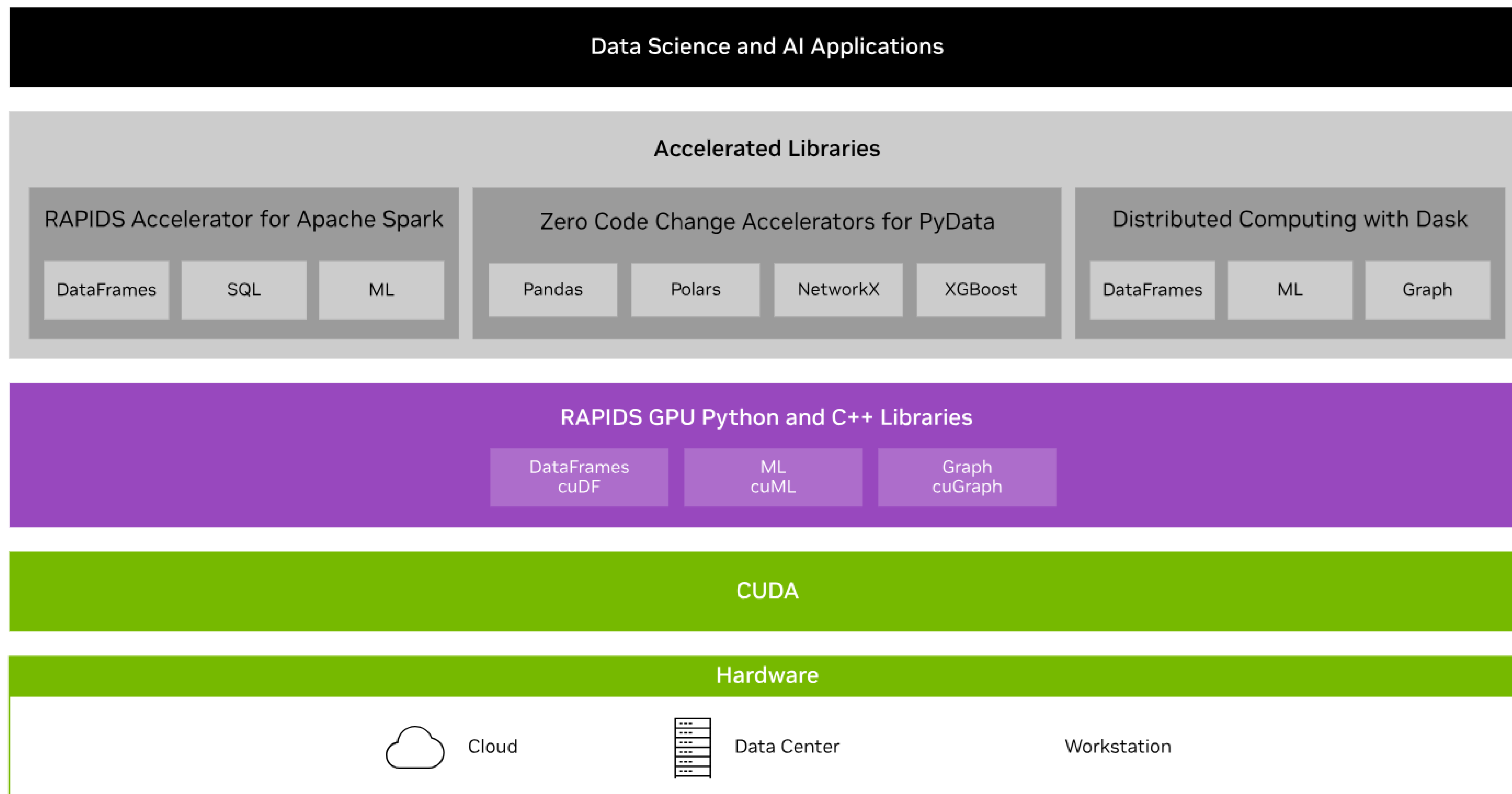
- Data quality and quantity directly impact model performance
 - High-quality data can be expensive to acquire
 - More data requires more compute and more efficient algorithms



- **Computational resources:** training models on large datasets requires significant computational power
- **Hyperparameter tuning:** finding optimal configurations for model hyperparameters is time-consuming
- **Continuous adaptation:** models may become less accurate as environment changes over time - need for continuous monitoring and retraining
- **Talent shortage:** lack of professionals with the right skills to develop systems effectively

RAPIDS Transforms Data Science

An optimized hardware-to-software stack for the entire data science pipeline



Machine Learning and AI Go Beyond Deep Learning

cuML + XGBoost libraries

- GPUs are ubiquitous in Deep Learning, but >90% of enterprises are primarily using Machine Learning
- Using CUDA ML primitives, the same matrix operations allow GPUs to be performant at Machine Learning as well
- cuML is the Python bindings to the C++/CUDA based cuML algorithms that uses cuDF Dataframes as input
- Dask enables distributed cuML model training
 - Can easily run multiple Dask workers per node to allow for an easier development model of one worker per GPU regardless of single node or multi node environment

Classification \ Regression	Decision Trees / Random Forests Linear/Lasso/Ridge Regression Logistic Regression K-Nearest Neighbors Support Vector Machine Classification
Inference	Random Forest / GBDT Inference
Clustering	K-Means DBSCAN Spectral Clustering
Decomposition & Dimensionality Reduction	Principal Components Singular Value Decomposition UMAP Spectral Embedding T-SNE
Time Series	Holt-Winters Seasonal ARIMA
Cross Validation	
Hyper-parameter Tuning	

Clustering

Group unlabeled examples based on their similarity to each other



Use cases:

- Market segmentation
- Anomaly detection
- *Imputation*

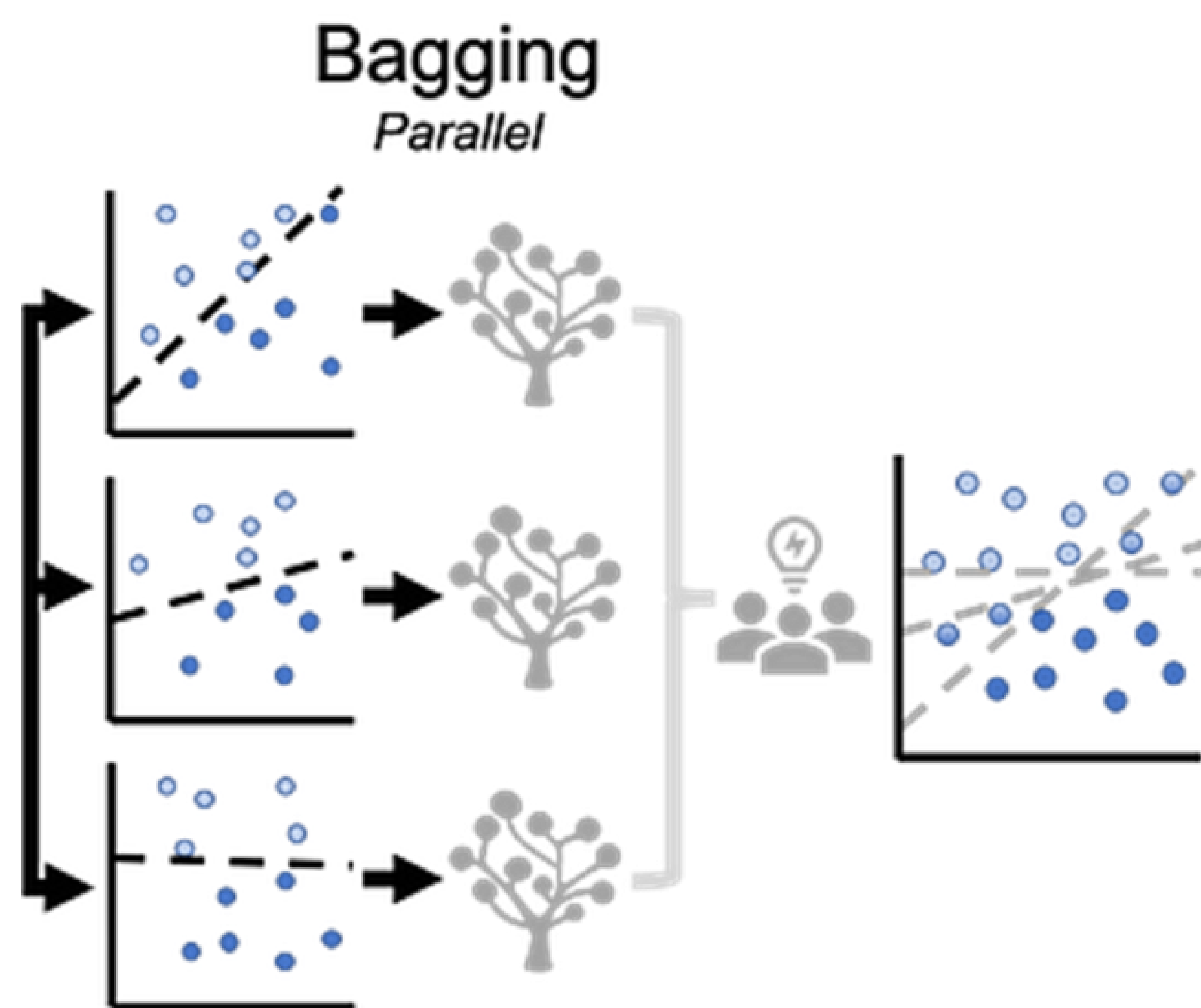
	K-Means	DBSCAN
Approach	Partitions data by iteratively moving cluster centers and re-assigning data points based on which center is closest to the point	Uses spatial density to cluster—finds cluster points within at most epsilon distance of another point in the cluster
Pros	● Fast, simple, and easy to understand ● Scalable to large datasets	● Can identify clusters of any arbitrary shapes ● Automatically determines the number of clusters ● Robust to outliers
Cons	● Assumes spherical clusters ● Requires specifying the number of clusters in advance ● Sensitive to outliers	● Less efficient for very large datasets ● Requires specifying minimum points and maximum neighborhood distance

Classification

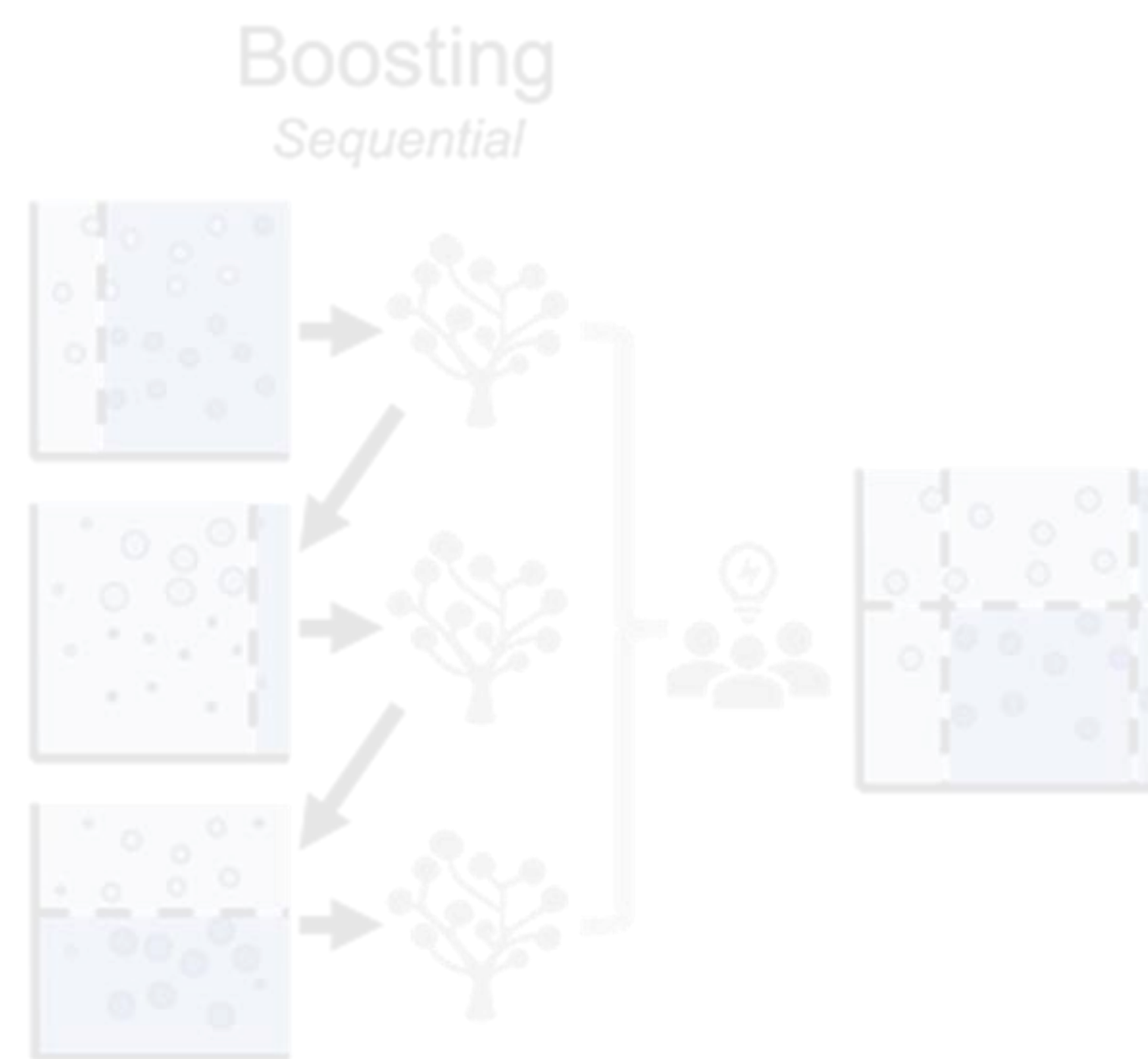
	K-Nearest Neighbors	Logistic Regression	Decision Tree
Approach	KNN makes predictions based on the similarity of new data points to known examples in the training set. The algorithm calculates the distance between the new point and all points in the training set. It identifies the K nearest neighbors and uses majority voting among the K neighbors	Logistic regression is a statistical method used for binary classification. It predicts the probability of an instance belonging to a particular class, using the logistic function (sigmoid) to map predictions to probabilities between 0 and 1	Creates a tree-like model of decisions based on features in the data. From the root node, it splits the data into subsets based on the most significant attributes. The splitting criteria is determined by maximizing information gain or minimizing impurity
Pros	• Makes no assumption about data distribution • “Lazy” learning algorithm • Adapts quickly to new data	• Generally efficient • Less prone to overfitting	• Does not require feature scaling • Robust to outliers • Can handle categorical variables • Makes no assumption about data distribution
Cons	• Prone to overfitting • Sensitive to outliers • Requires feature scaling • High computational cost for large datasets, especially during prediction • High memory requirement (needs to store the entire training dataset)	• Sensitive to outliers • Assumes linear relationship between features and log-odds of outcome • Suitable for binary outcome • Assumes no correlation between features • Requires irrelevant features to be excluded • Can overfit on high-dimensional datasets	• Prone to overfitting • Computationally expensive for large datasets

Ensemble

Using multiple decision trees



(Random Forest) Bagging is an ensemble technique used to reduce the variance of predictions by considering results from multiple decision tree models that were trained on different sub-samples of the same data set



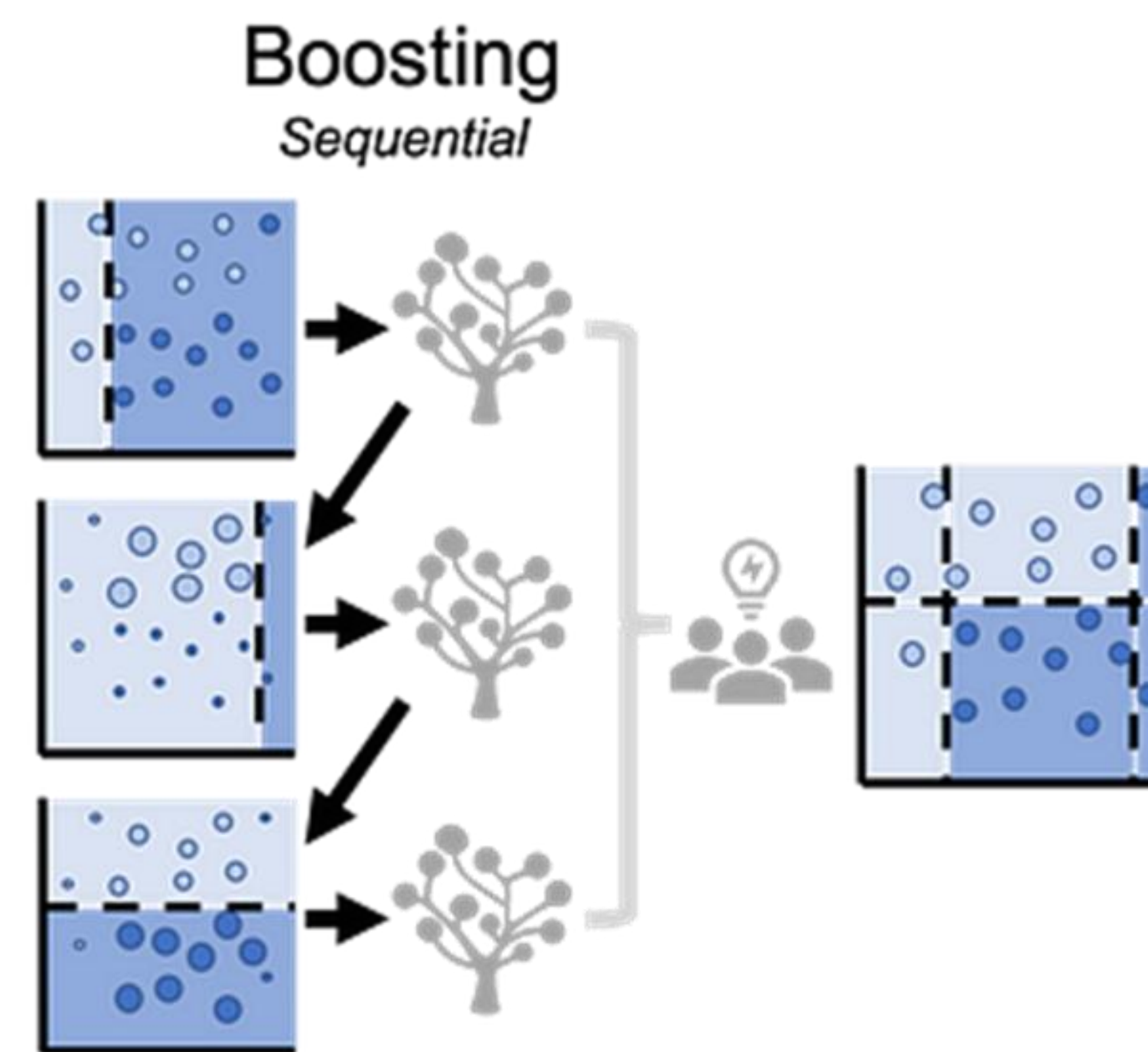
(Gradient Boosting Decision Tree) Boosting is another type of ensemble learning that combines weak learners to achieve improved model performance. Weak learners are simple models that predict relatively poorly. The concept of boosting is to sequentially train models, each time trying to fit better than before using residual errors

Ensemble

Using multiple decision trees



(Random Forest) Bagging is an ensemble technique used to reduce the variance of predictions by considering results from multiple decision tree models that were trained on different sub-samples of the same data set



(Gradient Boosting Decision Tree) Boosting is another type of ensemble learning that combines weak learners to achieve improved model performance. Weak learners are simple models that predict relatively poorly. The concept of boosting is to sequentially train models, each time trying to fit better than before using residual errors

XGBoost + RAPIDS ([LINK](#))

Optimized gradient boosting machine learning algorithm



- XGBoost is the gold standard in single model performance for classification and regression. RAPIDS was built from the start to integrate with this amazing algorithm.
- Extreme Gradient Boosting, is a scalable, distributed gradient-boosted decision tree machine learning algorithm
- Builds upon supervised machine learning, decision trees, ensemble learning, and gradient boosting concepts
- Gradient boosting is an extension of boosting where the process of additively generating weak models is formalized as a gradient descent algorithm over an objective function. Gradient boosting sets targeted outcomes for the next model to minimize errors
- With XGBoost, trees are built in parallel. It follows a level-wise strategy, scanning across gradient values and using these partial sums to evaluate the quality of splits at every possible split in the training set
- XGBoost includes seamless GPU acceleration, including loading data from cuDF DataFrames and cuPy arrays. This significantly speeds up model training and improves accuracy for better predictions. The Dask API makes it easy to scale to multiple nodes or multiple GPUs. Furthermore, the RAPIDS Memory Manager (RMM) integrates with XGBoost to enable a single, shared, high-speed memory pool

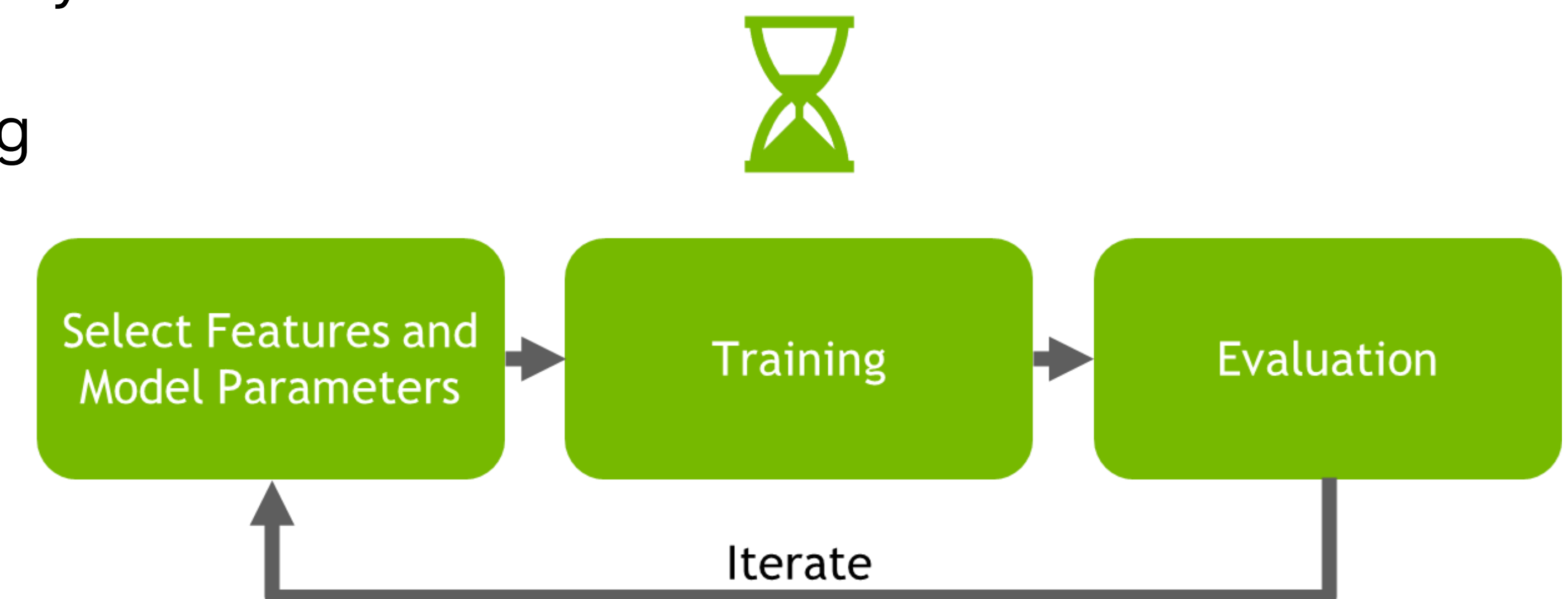


Acceleration

Model Tuning

Low data processing cycle time translates to more experimentation and higher productivity

- Faster runtime means more experimentation to improve model performance
- Faster runtime shortens development time and improves productivity
- **Cross validation** – resampling method to protect against overfitting
 - Trains multiple models with different subsets
- **Feature selection** – finding the optimal subset of features
 - Reduce noise and irrelevant features
 - Reduce model complexity and training time
 - Improve model inference performance
- Hyperparameter Tuning
 - **Grid Search**



Feature Engineering

Accelerated by RAPIDS cuDF

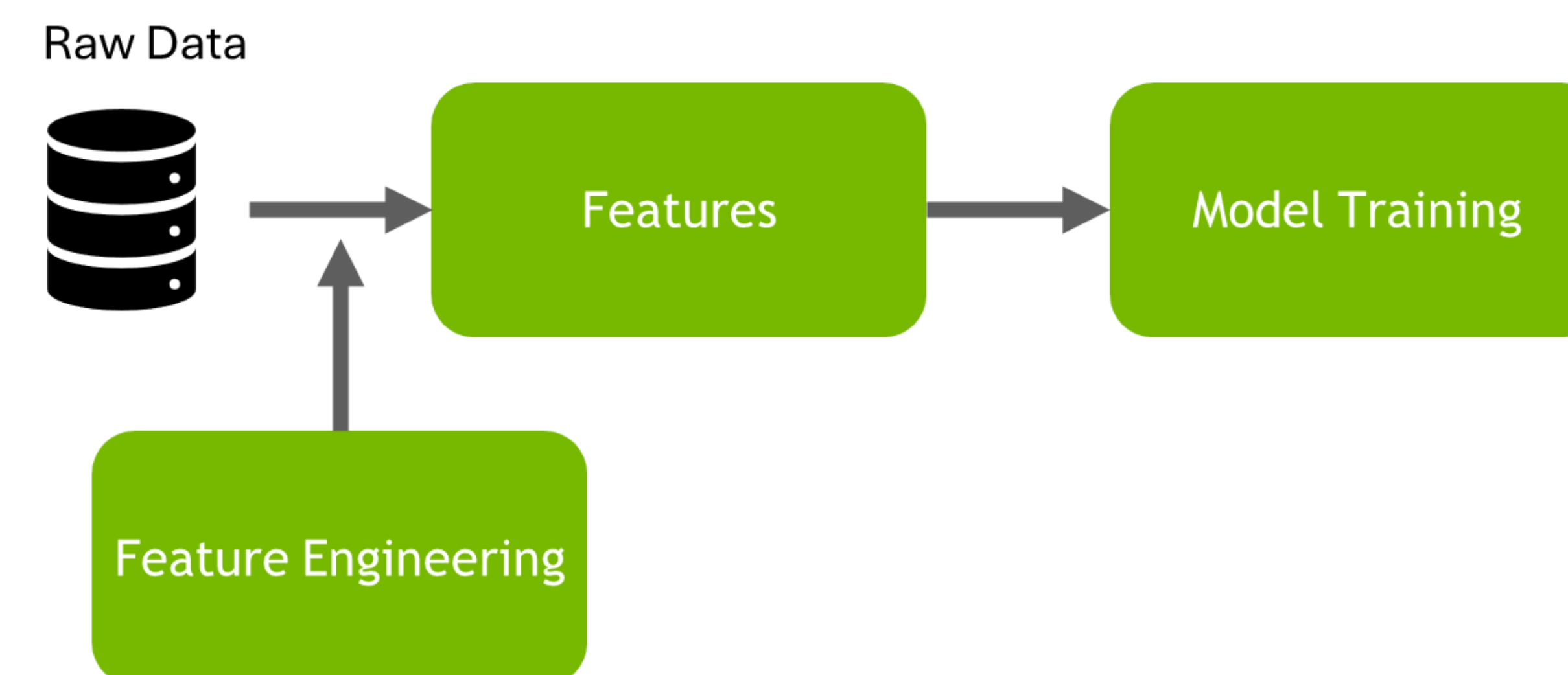
- **Feature engineering** is often considered one of the most crucial and impactful steps in the machine learning process to improve a model's predictive power and ability to generalize to new data

Benefits of feature engineering:

- Transforms raw data into a format that better represents the underlying problem, making it easier for algorithms to learn patterns and make predictions
 - Allows domain knowledge to be incorporated into the model, capturing important aspects of the problem that may not be immediately apparent in the raw data
 - Enhance model interpretability, making it easier to understand and explain model predictions
 - Reduce computational costs and improve model efficiency by providing more relevant and informative inputs
- Feature engineering is an iterative process that allows for continuous refinement and improvement of model inputs

Use cuDF to create new features for downstream ML tasks:

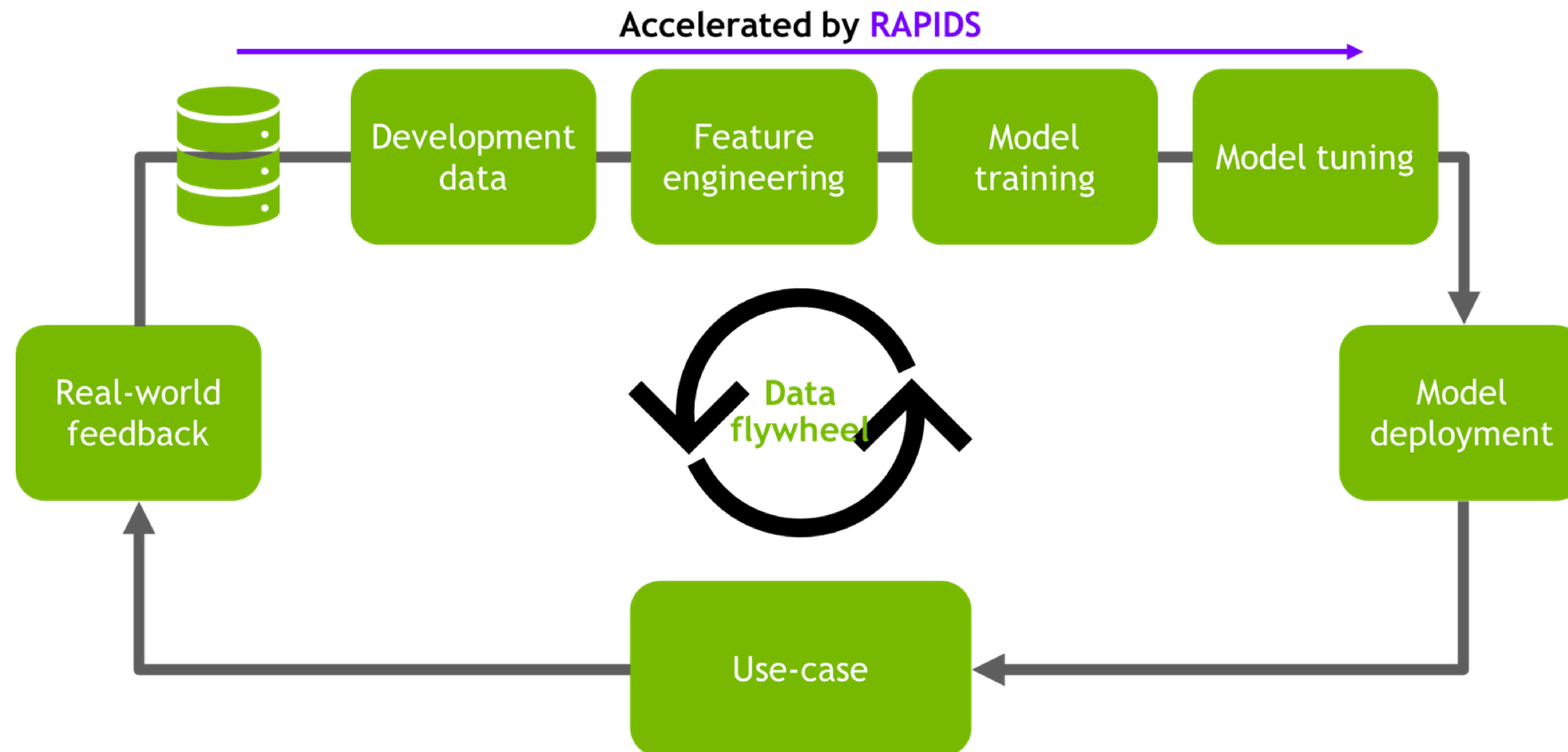
- cuML
- XGBoost
- cuDNN



Data Flywheel

Powerful approach to identifying and fixing data shifts

Enables systems to automatically improve their performance through the cyclical nature of data-driven growth and innovation

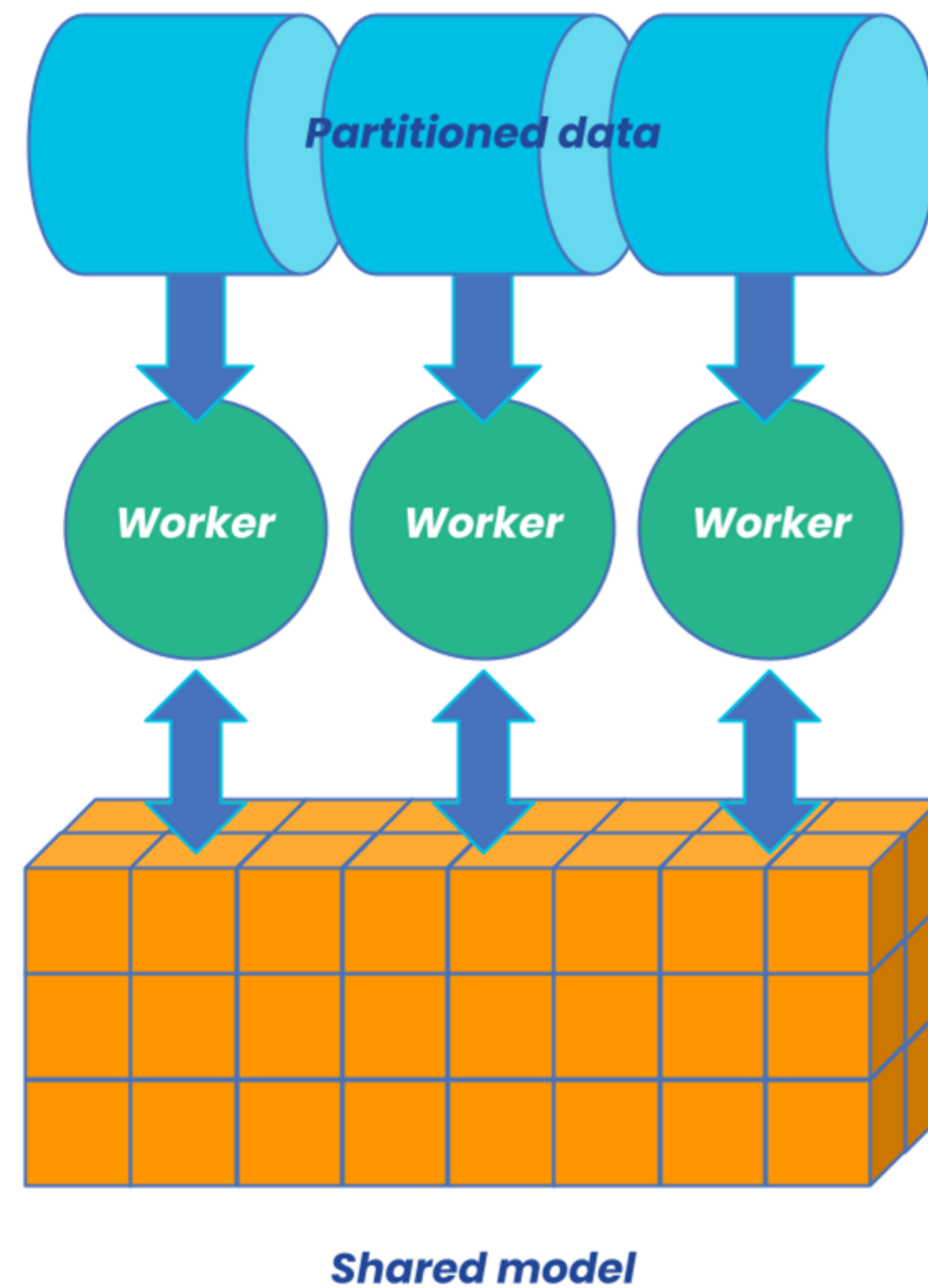


Distributed Computing for Machine Learning

When the training data or model doesn't fit in the memory of a single device

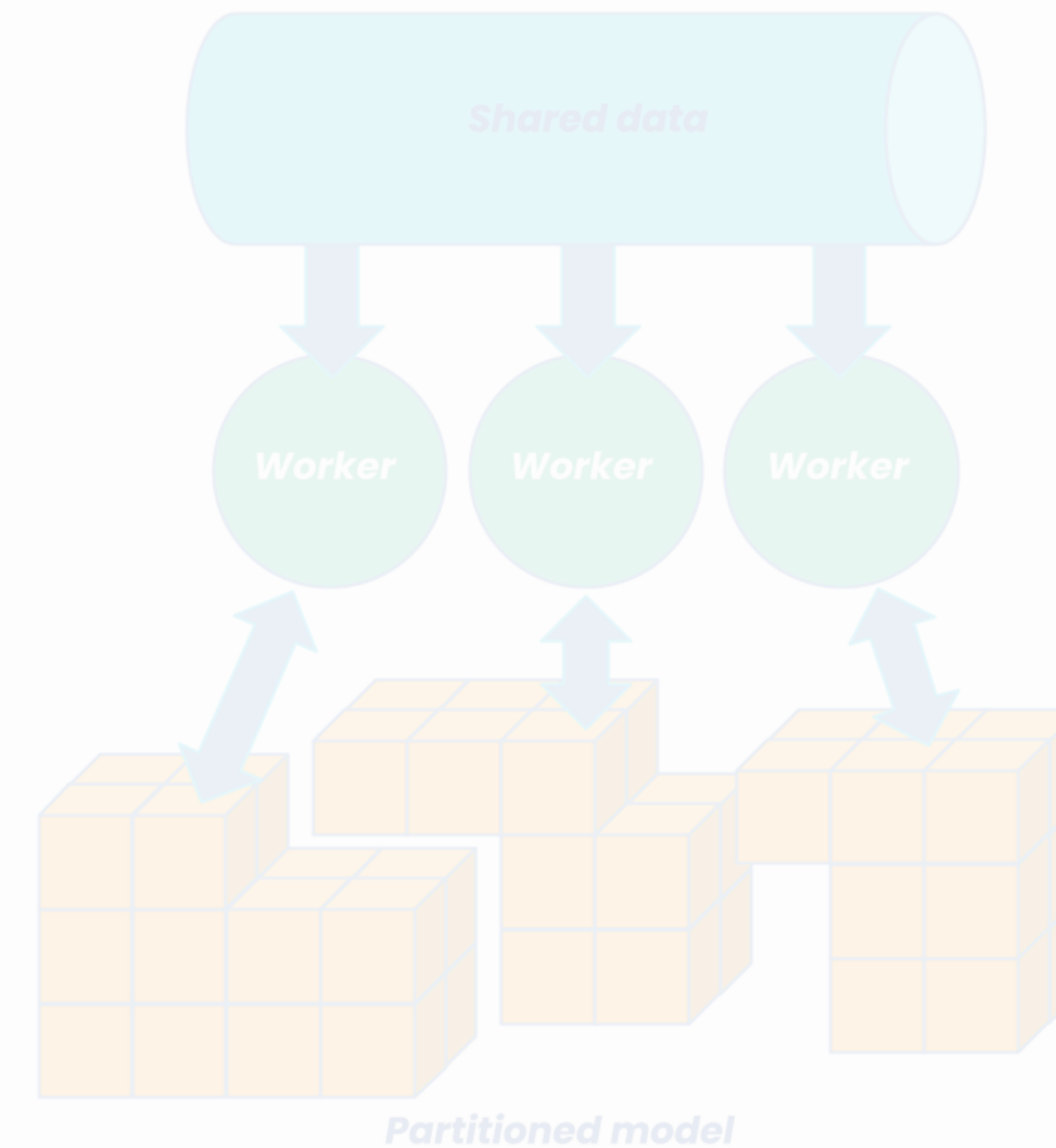
The dataset is split across multiple devices, with each device having a full copy of the model

Data parallelism



The model itself is split across multiple devices, with each device processing the same data but different parts of the model

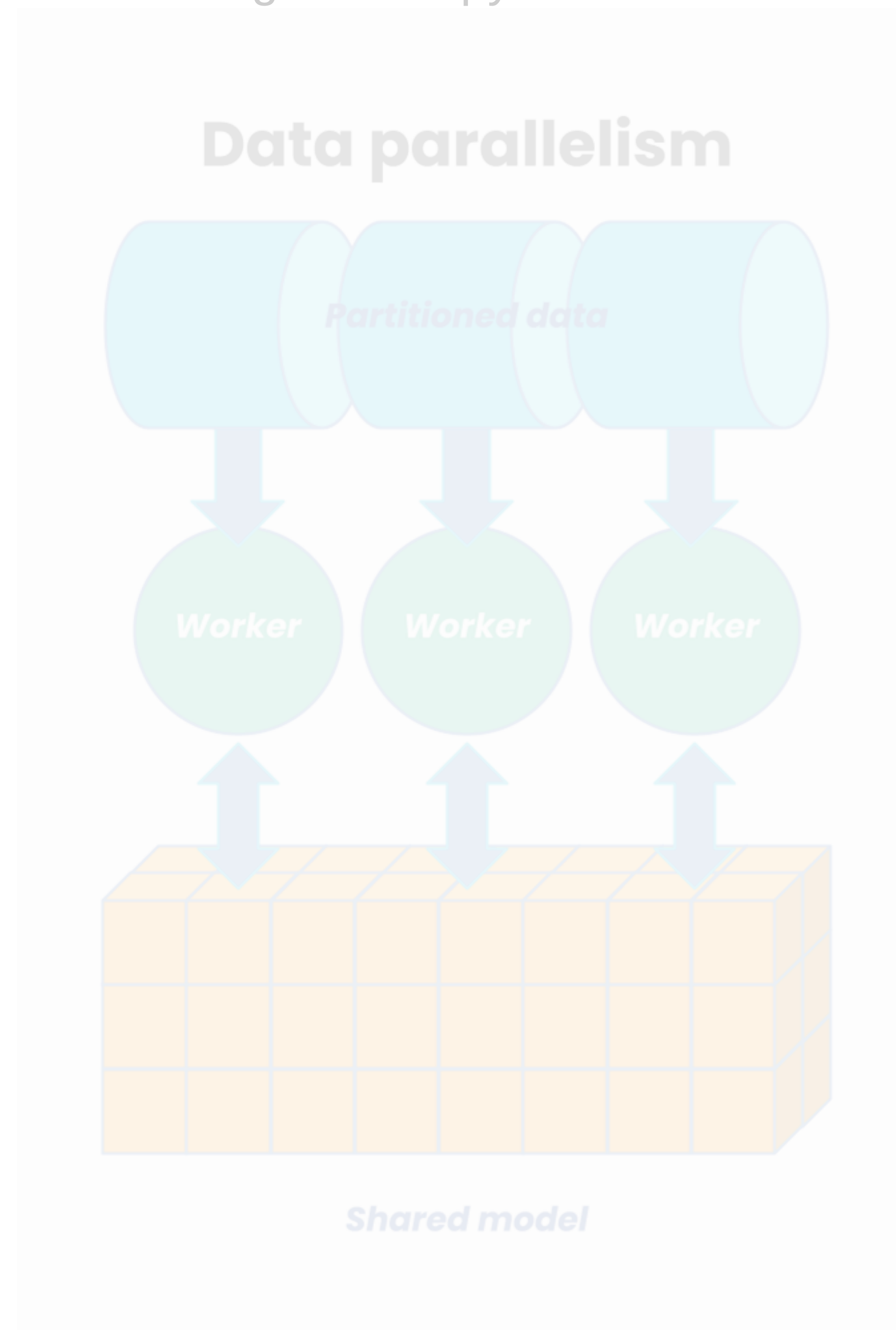
Model parallelism



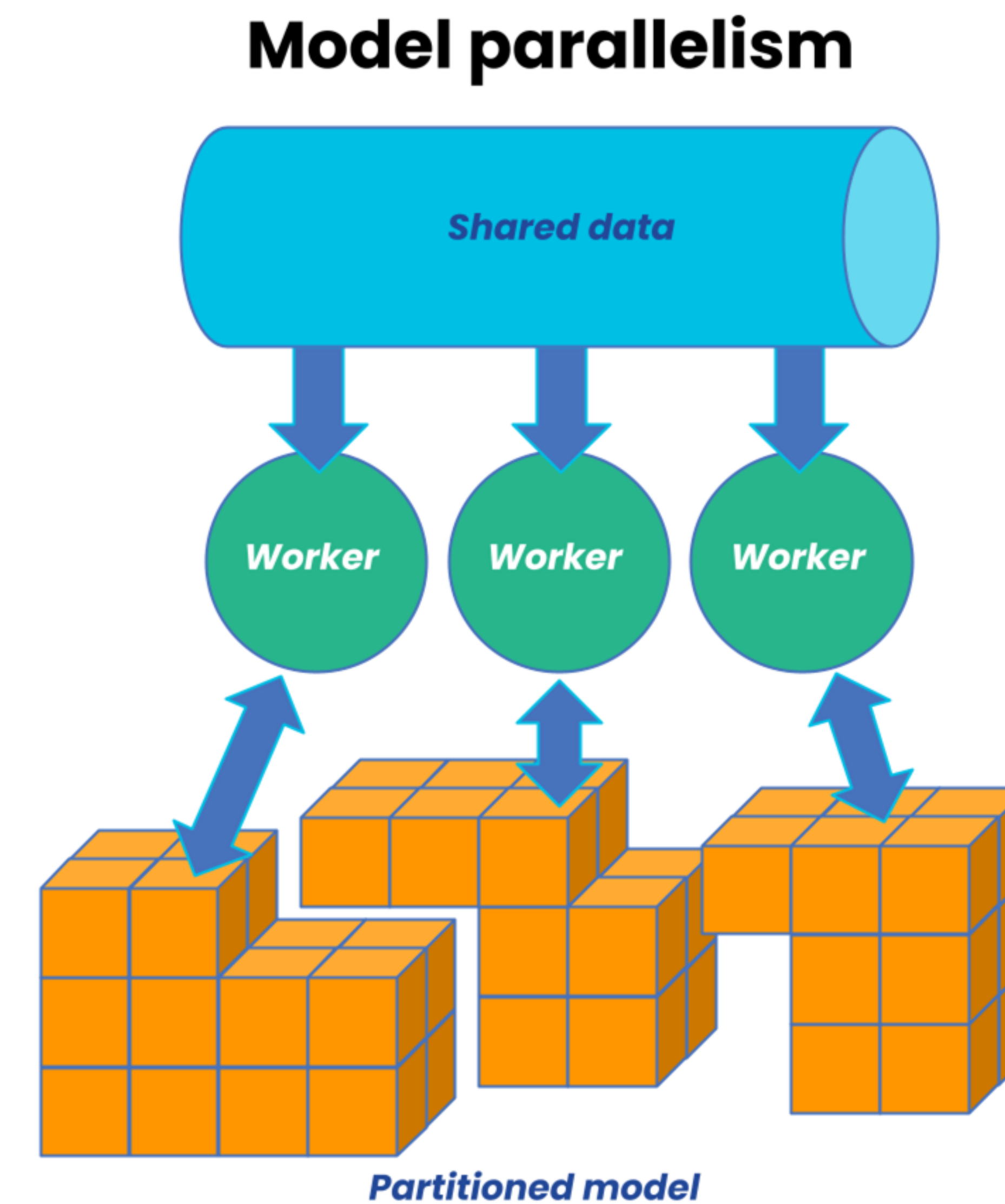
Distributed Computing for Machine Learning

When the training data or model doesn't fit in the memory of a single device

The dataset is split across multiple devices, with each device having a full copy of the model

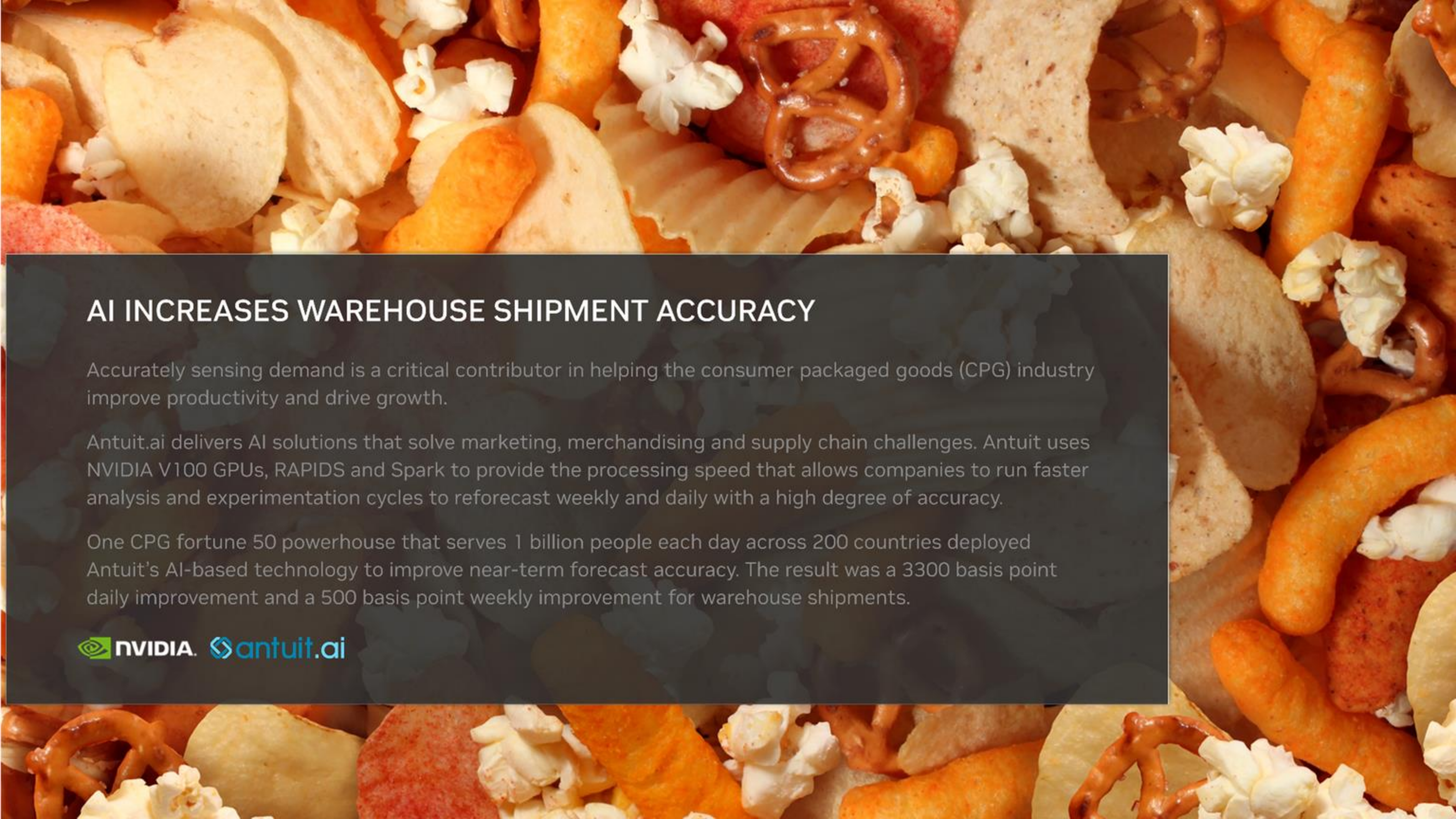


The model itself is split across multiple devices, with each device processing the same data but different parts of the model





Case Studies

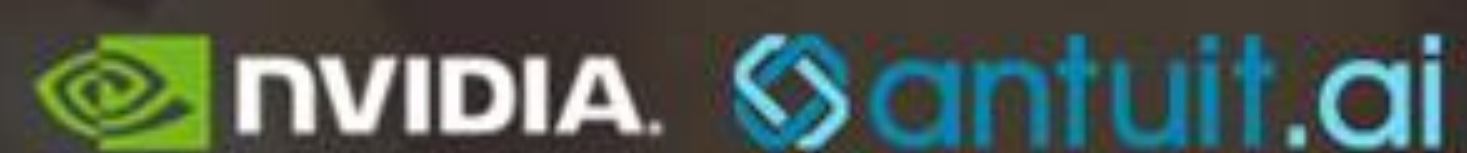


AI INCREASES WAREHOUSE SHIPMENT ACCURACY

Accurately sensing demand is a critical contributor in helping the consumer packaged goods (CPG) industry improve productivity and drive growth.

Antuit.ai delivers AI solutions that solve marketing, merchandising and supply chain challenges. Antuit uses NVIDIA V100 GPUs, RAPIDS and Spark to provide the processing speed that allows companies to run faster analysis and experimentation cycles to reforecast weekly and daily with a high degree of accuracy.

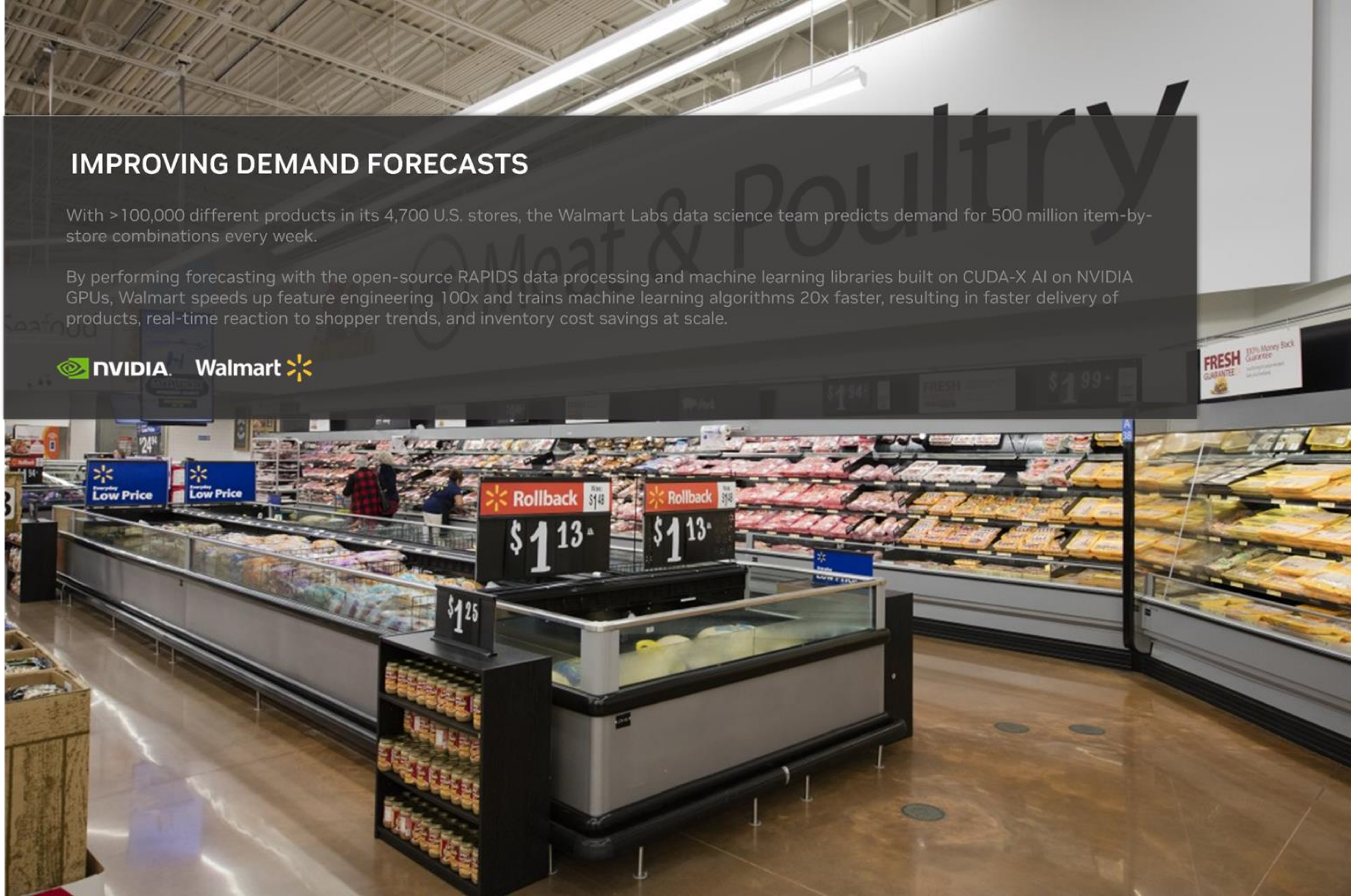
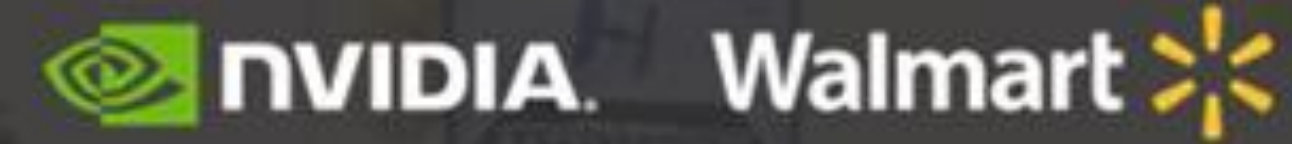
One CPG fortune 50 powerhouse that serves 1 billion people each day across 200 countries deployed Antuit's AI-based technology to improve near-term forecast accuracy. The result was a 3300 basis point daily improvement and a 500 basis point weekly improvement for warehouse shipments.



IMPROVING DEMAND FORECASTS

With >100,000 different products in its 4,700 U.S. stores, the Walmart Labs data science team predicts demand for 500 million item-by-store combinations every week.

By performing forecasting with the open-source RAPIDS data processing and machine learning libraries built on CUDA-X AI on NVIDIA GPUs, Walmart speeds up feature engineering 100x and trains machine learning algorithms 20x faster, resulting in faster delivery of products, real-time reaction to shopper trends, and inventory cost savings at scale.





IMPROVED INVENTORY FORECASTING

Fully stocked retail shelves are critical to retaining loyal customers, but accurate inventory forecasting is a challenging science requiring complex algorithms and massive amounts of data and compute cycles.

A mid-market retailer's legacy infrastructure was an I/O bottleneck. It took 10 minutes to pull in ~1M rows of data for 10 stores, or 1 store/minute on CPUs.

Using RAPIDS with cuDF, the retailer now pulls in millions of rows of data for 6,000 stores in under 3 minutes, or 2,000 stores/minute — a 2,000X speedup.



AI & DATA SCIENCE FOR NETWORK OPERATIONS

A wireless network operator had access to terabytes of data daily but needed an efficient way to gain insights from it.

A deep learning solution powered by NVIDIA DGX POD, RAPIDS, and software from Datalogue and OmniSci, changed the way they collect, process, visualize and understand data.

Data prep improved from 8 days to 4 minutes and the company's new AI models predict high-surge Wi-Fi usage and detect anomalies with 99% accuracy.



